

Data Scraping Project

Data Source:: Gofundme Official Website

Start Date: Wed 3rd July 2024

Finished Date: Fri 5th July 2024

Environment: Jupyter Colab- Python 3(ipynbkernel)|Idle

Overview

This script uses Selenium and BeautifulSoup to automatically scrape data from GoFundMe campaigns related to "Gaza". It extracts various details such as campaign titles, links, organizer names, amounts raised, goals, number of donations, and descriptions. It also includes a mechanism to save progress and resume from where it left off in case of an interruption.

Load Libraries needed to perform all the Operations of Data Collection/Scraping

```
In [1]: import subprocess
import sys

# Function to install a package if it's not already installed
def install_and_import(package):
    try:
        __import__(package)
    except ImportError:
        print(f"{package} not found. Installing...")
        subprocess.check_call([sys.executable, "-m", "pip", "install", package])
    finally:
        globals()[package] = __import__(package)

# List of packages to check and install if necessary
packages = [
    "selenium",
    "bs4",
    "pandas",
]

# Dictionary to map import names to package names (for cases where they differ)
package_import_map = {
    "selenium": ["webdriver", "webdriver.edge.service", "webdriver.common.by", "webdriver.support.ui", "webdriver"],
    "bs4": ["BeautifulSoup"],
    "pandas": ["pd"],
    "time": [],
    "json": [],
    "os": [],
}

# Install and import the packages
for package in packages:
    install_and_import(package)

# Import other standard libraries directly as they don't need to be installed
import time
import json
import os

# Import submodules and rename if necessary
from selenium import webdriver
from selenium.webdriver.edge.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from bs4 import BeautifulSoup
import pandas as pd

print("All required libraries are installed and imported.")
```

All required libraries are installed and imported.

Explanation of the Code Above:

- `install_and_import(package)`:

This function tries to import a package. If the package is not found (`ImportError`), it installs the package using `subprocess.check_call`. Finally, it imports the package into the global namespace.

- `packages`:

A list of packages that need to be checked and potentially installed.

- `package_import_map`:

A dictionary mapping package names to their respective import names (useful if the import name is different from the package name).
Loop:

Iterates through the list of packages, installing and importing each one.

- `Standard Libraries`:

Imports the `time`, `json`, and `os` libraries directly as they are part of the Python standard library and do not need to be installed.

- `Submodule Imports`:

Imports submodules from the `selenium` and `bs4` packages to make them available for use in your code.

- By running this script, it ensures that all required libraries are installed and imported, allowing for one to proceed with the main script without any interruption.

```
In [21]: # Specify the path to the Edge WebDriver executable
edge_driver_path = r'C:\Users\USER\Downloads\edgedriver_win64\msedgedriver.exe'

# Set up the Edge WebDriver service
service = Service(edge_driver_path)

# Initialize the WebDriver with retry mechanism
def init_driver():
    options = webdriver.EdgeOptions()
    options.add_argument('headless')
    driver = webdriver.Edge(service=service, options=options)
    return driver

driver = init_driver()

# Open the GoFundMe search page for Gaza
url = 'https://www.gofundme.com/s?q=gaza'
driver.get(url)

# Click the "Show more" button until all campaigns are loaded
while True:
    try:
        show_more_button = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.XPATH, '//button[@data-element-id="btn_show_more"]')))
        show_more_button.click()
        time.sleep(5) # Allow time for the new campaigns to load
    except:
        # Exit the loop if the "Show more" button is no longer found
        break

# Get the page source after loading all campaigns
page_source = driver.page_source

# Parse the page source with BeautifulSoup
soup = BeautifulSoup(page_source, 'html.parser')

# Extract campaign links
campaign_divs = soup.find_all('a', class_='full-state-list-card_actionCard_c6v8R hrt-action-card hrt-base-buttn')
campaign_links = ['https://www.gofundme.com/' + a['href'] for a in campaign_divs]

# Extract campaign titles and organizer names
campaign_data = []
divs_titles = soup.find_all('div', class_='full-state-list-card_truncate_79X_9 full-state-list-card_fundName_')
divs_organizers = soup.find_all('div', class_='full-state-list-card_organizer_j97E8 full-state-list-card_trunc')

for i in range(len(campaign_links)):
    title = divs_titles[i].get_text() if i < len(divs_titles) else ""
    organizer = divs_organizers[i].find('span').get_text().strip() if i < len(divs_organizers) else ""
```

```

campaign_data.append({
    'Campaign Link': campaign_links[i],
    'Campaign Title': title,
    'Organizer Name': organizer,
    'Amount Raised': "",
    'Goal': "",
    'Number of Donations': "",
    'Description': ""
})

# Define the full path to the progress file
progress_file_path = r'C:\Users\USER\Documents\progress_gofundme2.json'

# Load saved progress if exists and valid
if os.path.exists(progress_file_path):
    try:
        with open(progress_file_path, 'r') as file:
            progress = json.load(file)
            campaign_data = progress.get('campaign_data', campaign_data)
            start_index = progress.get('last_index', -1) + 1 # Start from the next campaign
            print(f"Loaded progress: start_index={start_index}")
    except (json.JSONDecodeError, KeyError) as e:
        print(f"Error loading progress file: {e}")
        # If there's an error loading the progress file, start from scratch
        start_index = 0
else:
    start_index = 0

# Iterate over each campaign starting from the last saved index
for index in range(start_index, len(campaign_data)):
    link = campaign_data[index]['Campaign Link']
    try:
        # Open the campaign page
        driver.get(link)

        # Wait for the campaign page to load by checking for the presence of the amount raised element
        WebDriverWait(driver, 20).until(
            EC.presence_of_element_located((By.CLASS_NAME, 'hrt-disp-inline'))
        )

        # Retry mechanism for clicking the "Read more" button
        retry_count = 4
        for attempt in range(retry_count):
            try:
                read_more_button = WebDriverWait(driver, 10).until(
                    EC.element_to_be_clickable((By.CSS_SELECTOR, 'button[data-element-id="btn_story_read-more"]'))
                )
                read_more_button.click()
                WebDriverWait(driver, 10).until(
                    EC.staleness_of(read_more_button)
                )
                break # Exit the loop if the click is successful
            except Exception as e:
                if attempt < retry_count - 1:
                    print(f"Retrying ({attempt+1}/{retry_count})...")
                    time.sleep(5) # Wait before retrying
                else:
                    print(f"Read more button not found or not clickable: {e}")

        # Get the page source of the campaign
        campaign_page_source = driver.page_source

        # Parse the campaign page source with BeautifulSoup
        campaign_soup = BeautifulSoup(campaign_page_source, 'html.parser')

        # Extract the required data
        amount_raised_text = campaign_soup.find('div', class_='hrt-disp-inline').get_text().strip() if campaign
        goal_text = campaign_soup.find('span', class_='hrt-text-body-sm hrt-text-gray').get_text().strip() if ca
        donations_no_text = campaign_soup.find('span', class_='hrt-text-gray-dark').get_text().strip() if campa

        # Store the extracted data in the dictionary
        campaign_data[index]['Amount Raised'] = amount_raised_text
        campaign_data[index]['Goal'] = goal_text
        campaign_data[index]['Number of Donations'] = donations_no_text

        # Extract the description text
        description_div = campaign_soup.find('div', class_='campaign-description_content_C1C_5 hrt-mt-3 campai
        description_text = ''
        if description_div:
            for child in description_div.find_all(recursive=False):
                if child.name == 'div' and 'img' not in child.attrs:
                    description_text += child.get_text(separator=' ', strip=True) + ' '

```

```

campaign_data[index]['Description'] = description_text.strip()

# Save progress periodically
if index % 10 == 0:
    with open(progress_file_path, 'w') as file:
        json.dump({
            'campaign_data': campaign_data,
            'last_index': index
        }, file)
    print(f"Progress saved at index {index}")

except Exception as e:
    print(f"An error occurred: {e}")
    # Reinitialize the driver and continue
    driver.quit()
    driver = init_driver()
    continue

# Save final progress
with open(progress_file_path, 'w') as file:
    json.dump({
        'campaign_data': campaign_data,
        'last_index': len(campaign_data) - 1
    }, file)

# Close the browser
driver.quit()

# Convert the list of dictionaries to a DataFrame
df = pd.DataFrame(campaign_data)

# Save the DataFrame to a CSV file
df.to_csv(r'C:\Users\USER\Documents\gofundme_campaigns11.csv', index=False)

print("Data saved to CSV.")

```

Loaded progress: start_index=1000
Data saved to CSV.

In [22]: display(df)

	Campaign Link	Campaign Title	Organizer Name	Amount Raised	Goal	Number of Donations	Description
0	https://www.gofundme.com/f/help-rabeha-and-her...	Help Rabeha and her family leave Gaza before i...	rabeha alzayegh	\$248,187	USD raised of \$250,000 goal	12.2K	Hi world , it's Rabeha Please read this as if...
1	https://www.gofundme.com/f/help-alaas-family-e...	Help Alaa's Family Escape Gaza & Live a Safe Life	MOHAMMED abu rezeq	€27,569	raised of €43,000 goal	1.2K	Hello to all lovers of life and givers of hope...
2	https://www.gofundme.com/f/support-emans-famil...	Support Eman's Family and Gaza Community	Liza Berrospi	\$46,530	USD raised of \$50,000 goal	984	Update: Gofundme has finally verified my accou...
3	https://www.gofundme.com/f/support-mohameds-fa...	Support Mohamed's family to reach safety outsi...	Ibrahim Alshayyah	\$18,011	USD raised of \$30,000 goal	1.4K	Dear friends, my name is Mohamed Alshayyah fro...
4	https://www.gofundme.com/f/mp95k-evacuate-my-f...	Evacuate my family from Gaza	Samah Family	€17,475	raised of €50,000 goal	1.3K	I am Samah, and my situation is like everyone ...
...	...	...	...	...	...	...	...
995	https://www.gofundme.com/f/Liberta-alla-popola...	Aiutiamo i bambini di Gaza e le loro famiglie	Gianna Pasini	€2,645	raised of €10,000 goal	60	Gaza: dentro la tenda dove vive la famiglia di...
996	https://www.gofundme.com/f/kmgzjg-help-rebuild...	Help Rebuild Muhammad's Home in Gaza	Calla Walsh	\$7,486	USD raised of \$15,000 goal	113	Hi. I am Muhammad Ibrahim from the Gaza Strip....
997	https://www.gofundme.com/f/contribution-to-rem...	Help Rama who has down syndrome and her family...	Mohammed Albordaini	€4,454	raised of €25,000 goal	254	Hi Please help us arrange the transportation o...
998	https://www.gofundme.com/f/support-kamels-fami...	Return of hope to family from gaza	Dina Nawaf	\$194	AUD raised of \$30,000 goal	11	Hi everyone!! A new campaign worked, an old on...
999	https://www.gofundme.com/f/d597fb-help-my-fami...	Help my Family get out of Gaza	AMEER SHAHEEN	€1,560	raised of €31,000 goal	111	Dear friends and good people, My name is Amir...

1000 rows × 7 columns

References for the Data Source

GoFundMe , G. (n.d.). GoFundMe search: Gaza. <https://www.gofundme.com/s?q=gaza>

```
In [ ]:
In [ ]:
```

GoFundMe Campaign Data Scraper

This documentation provides a step-by-step explanation of the GoFundMe campaign data scraper. The scraper is designed to extract information about GoFundMe campaigns for Gaza, including campaign links, titles, organizer names, amounts raised, goals, number of donations, and descriptions.

Prerequisites

Before running the code, ensure you have the following:

- Python installed on your machine.
- Required Python libraries: `re` , `pandas` , `webdriver` , `BeautifulSoup` , `time` , `json` , `os` .

Overview

The script performs the following tasks:

1. Set up the Edge WebDriver service.
2. Initialize the WebDriver with a retry mechanism.
3. Open the GoFundMe search page for Gaza.
4. Load all campaigns by clicking the "Show more" button until no more campaigns are available.
5. Extract campaign data such as links, titles, and organizer names.
6. Save the progress periodically and load the saved progress if available.
7. Iterate over each campaign to extract detailed information like amounts raised, goals, number of donations, and descriptions.
8. Save the extracted data to a CSV file.

Detailed Explanation

Step 1: Specify the Path to the Edge WebDriver Executable

```
edge_driver_path = r'C:\Users\USER\Downloads\edgedriver_win64\msedgedriver.exe'
service = Service(edge_driver_path)
```

Specify the path where the Edge WebDriver executable is located and set up the Edge WebDriver service.

Step 2: Initialize the WebDriver

```
def init_driver():
    options = webdriver.EdgeOptions()
    options.add_argument('headless')
    driver = webdriver.Edge(service=service, options=options)
    return driver
```

```
driver = init_driver()
```

Define a function to initialize the Edge WebDriver with headless options, which allows the browser to run in the background without a graphical user interface.

Step 3: Open the GoFundMe Search Page for Gaza

```
url = 'https://www.gofundme.com/s?q=gaza'
driver.get(url)
```

Navigate to the GoFundMe search page for Gaza.

Step 4: Load All Campaigns

```
while True:
    try:
        show_more_button = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.XPATH, '//button[@data-element-id="btn_show_more"]')))
        show_more_button.click()
        time.sleep(5)
    except:
        break
```

Click the "Show more" button repeatedly until all campaigns are loaded. The loop continues until the "Show more" button is no longer found.

Step 5: Extract Campaign Data

```
page_source = driver.page_source
soup = BeautifulSoup(page_source, 'html.parser')
campaign_divs = soup.find_all('a', class_='full-state-list-card_actionCard__c6v8R hrt-action-card hrt-base-button')
campaign_links = ['https://www.gofundme.com' + a['href'] for a in campaign_divs]

divs_titles = soup.find_all('div', class_='full-state-list-card_truncate__79X_9 full-state-list-card_fundName__oq_nD')
divs_organizers = soup.find_all('div', class_='full-state-list-card_organizer__j97E8 full-state-list-card_truncateSingle__AtDr_ hrt-text-body-sm hrt-text-gray hrt-font-regular')

campaign_data = []
for i in range(len(campaign_links)):
    title = divs_titles[i].get_text() if i < len(divs_titles) else ""
    organizer = divs_organizers[i].find('span').get_text().strip() if i < len(divs_organizers) else ""

    campaign_data.append({
        'Campaign Link': campaign_links[i],
        'Campaign Title': title,
        'Organizer Name': organizer,
        'Amount Raised': "",
```

```

        'Goal': "",
        'Number of Donations': "",
        'Description': ""
    })

```

Extract campaign links, titles, and organizer names from the loaded page source using BeautifulSoup.

Step 6: Save and Load Progress

```

progress_file_path = r'C:\Users\USER\Documents\progress_gofundme2.json'

```

```

if os.path.exists(progress_file_path):
    try:
        with open(progress_file_path, 'r') as file:
            progress = json.load(file)
            campaign_data = progress.get('campaign_data', campaign_data)
            start_index = progress.get('last_index', -1) + 1
    except (json.JSONDecodeError, KeyError) as e:
        start_index = 0
else:
    start_index = 0

```

Save progress to a JSON file periodically and load saved progress if it exists to continue from where the script left off in case of interruptions.

Step 7: Extract Detailed Information for Each Campaign

```

for index in range(start_index, len(campaign_data)):
    link = campaign_data[index]['Campaign Link']
    try:
        driver.get(link)
        WebDriverWait(driver, 20).until(
            EC.presence_of_element_located((By.CLASS_NAME, 'hrt-disp-inline'))
        )

        retry_count = 4
        for attempt in range(retry_count):
            try:
                read_more_button = WebDriverWait(driver, 10).until(
                    EC.element_to_be_clickable((By.CSS_SELECTOR, 'button[data-element-
id="btn_story_read-more"]'))
                )
                read_more_button.click()
                WebDriverWait(driver, 10).until(
                    EC.staleness_of(read_more_button)
                )
                break
            except Exception as e:
                if attempt < retry_count - 1:
                    time.sleep(5)
                else:
                    continue

        campaign_page_source = driver.page_source
        campaign_soup = BeautifulSoup(campaign_page_source, 'html.parser')

        amount_raised_text = campaign_soup.find('div', class_='hrt-disp-inline').get_text().strip()
        if campaign_soup.find('div', class_='hrt-disp-inline') else ""
        goal_text = campaign_soup.find('span', class_='hrt-text-body-sm hrt-text-
gray').get_text().strip() if campaign_soup.find('span', class_='hrt-text-body-sm hrt-text-gray')
        else ""
        donations_no_text = campaign_soup.find('span', class_='hrt-text-gray-
dark').get_text().strip() if campaign_soup.find('span', class_='hrt-text-gray-dark') else ""

        campaign_data[index]['Amount Raised'] = amount_raised_text
        campaign_data[index]['Goal'] = goal_text
        campaign_data[index]['Number of Donations'] = donations_no_text

        description_div = campaign_soup.find('div', class_='campaign-description_content__C1C_5 hrt-
mt-3 campaign-description_isOpen__4cQeG')
        description_text = ''
        if description_div:
            for child in description_div.find_all(recursive=False):
                if child.name == 'div' and 'img' not in child.attrs:
                    description_text += child.get_text(separator=' ', strip=True) + ' '

        campaign_data[index]['Description'] = description_text.strip()

        if index % 10 == 0:

```

```

        with open(progress_file_path, 'w') as file:
            json.dump({
                'campaign_data': campaign_data,
                'last_index': index
            }, file)

    except Exception as e:
        driver.quit()
        driver = init_driver()
        continue

```

Iterate over each campaign link to extract detailed information such as amounts raised, goals, number of donations, and descriptions. Retry mechanisms and progress saving ensure the script's resilience and continuity.

Step 8: Save Data to CSV File

```

with open(progress_file_path, 'w') as file:
    json.dump({
        'campaign_data': campaign_data,
        'last_index': len(campaign_data) - 1
    }, file)

driver.quit()

df = pd.DataFrame(campaign_data)
df.to_csv(r'C:\Users\USER\Documents\gofundme_campaigns11.csv', index=False)

print("Data saved to CSV.")

```

Save the final progress and convert the list of dictionaries to a DataFrame, then save it to a CSV file.

Conclusion

This script effectively scrapes data from GoFundMe campaigns related to Gaza, ensuring data accuracy and continuity through retry mechanisms and periodic progress saving. The final output is a CSV file containing detailed campaign information.

In []:

In []:

In []:

NB: This code has been written to specifically scrape data available to the public from the GoFundMe website only and it follows the ethical guidelines for data privacy standards as per the GoFundMe terms of service.

© 2024 Philip Kibet

In []:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js